DEFINITION *SOLID* is a mnemonic for “single responsibility principle, open-closed, Liskov substitution, interface segregation, and dependency inversion.” This course by Steve Smith introduces the principles using C#: http://mng.bz/Ox1R.

On that basis, ASP.NET Core has DI (sometimes called *dependency inversion* or *inversion of control [IoC]*) baked into the heart of the framework. Regardless of whether you want to use DI within your own application code, the framework libraries themselves depend on it as a concept.

NOTE Although related, dependency injection and dependency inversion are two different things. I cover both in a general sense in this chapter, but for a good explanation of the differences, see this post by Derick Bailey, titled “Dependency Injection Is NOT the Same As the Dependency Inversion Principle”: http://mng.bz/5jvB.

When you started programming, chances are that you didn’t use a DI framework immediately. That’s not surprising or even a bad thing; DI adds a certain amount of extra wiring that’s often not warranted in simple applications or when you’re getting started. But when things start to get more complex, DI comes into its own as a great tool to help keep that complexity under control.

Let’s consider a simple example, written without any sort of DI. Suppose that a user has registered on your web app, and you want to send them an email. This listing shows how you might approach this task initially, using a minimal API endpoint handler.

Listing 8.1 Sending an email without DI when there are no dependencies

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```